1. Big picture: paano tayo nakikilala?

Taglish technical guide for your presentation. Based sa repository code reviewed on 12 October 2026. Ang line numbers ay para sa code snapshot na ito; puwedeng gumalaw kapag may edits.

Ang short answer

Ang Biz-Z ay may **email-and-password authentication**. React ang kumukuha ng input, Express ang nagva-validate, MongoDB ang nagse-save ng account, bcrypt ang nagha-hash ng password, at JWT ang ginagamit para sa session. Nasa **HttpOnly cookie** ang token, hindi sa localStorage.

```
React form -> axios -> Express /api/auth/login
           -> MongoDB user lookup -> bcrypt.compare
           -> signed JWT -> bizz_session cookie
Next request -> cookie -> jwt.verify -> req.user -> owned data
```

Authentication vs authorization

Authentication: "Sino ka?" Valid ba ang login/session? **Authorization:** "Aling records ang puwede mong makita?" Kahit logged in, hindi ka dapat makabasa ng ibang user's plan. Magkaibang checks ito.

Main files na ipapakita sa panel

client/src/Editor.tsx: AuthModal at submit handler.
client/src/services/api.ts: shared axios client.
server/routes/api.js: register, login, cookie, JWT middleware.
server/models/index.js: User schema.
server/middleware/http.js: origins at error responses.

Code reference: server/routes/api.js:76-130; client/src/services/api.ts:1-7; server/models/index.js:26-39

Presentation line: "Hindi sapat na may login screen. Ang actual verification ay nasa backend, at bawat protected request chine-check ulit gamit ang signed session cookie."

2. Registration: mula form hanggang account

1. Input validation sa frontend

React Hook Form plus Zod ang gamit ng AuthModal. Email must be valid; password must be 8-100 characters. Required ang name kapag register. Frontend validation gives immediate feedback, pero puwede itong ma-bypass, kaya inuulit ang validation sa server.

Code reference: client/src/schemas/forms.ts:2-7; client/src/Editor.tsx:662-710

2. POST /api/auth/register

Ang axios base URL ay /api by default. Kaya ang /auth/register na call sa component ay nagiging /api/auth/register. Server credentials schema validates the body, lowercases the email, at trims the name. Registration extends the schema para required ang name.

Code reference: client/src/services/api.ts:2-6, 16-27; server/routes/api.js:49-57, 94-97

3. Password hash, hindi plain password

```
passwordHash: await bcrypt.hash(data.password, 12)
```

Ang 12 ay bcrypt cost factor, hindi password length at hindi "12 repetitions" lang. Bcrypt generates a salt at gumagawa ng one-way hash. Walang decrypt-password step sa login. Ang saved field ay passwordHash, hindi password.

Code reference: server/routes/api.js:98-102; server/models/index.js:38

4. Unique email + automatic sign-in

May unique database index ang email. Kapag duplicate record, ang error handler returns 409. Pag successful ang creation, session(res, user) agad at 201 response. Ibig sabihin, signed in na ang bagong account. publicUser returns id, name, email, profile only; hindi kasama ang hash.

Code reference: server/models/index.js:31-38; server/middleware/http.js:56-57; server/routes/api.js:88-104

Panel answer: "We store a salted bcrypt hash. Even our normal API response does not include the password hash."

3. Login at session cookie

1. Lookup and comparison

```
const user = await User.findOne({ email: data.email });
if (!user || !(await bcrypt.compare(
  data.password, user.passwordHash)))
  return res.status(401).json({
    error: "Email or password is incorrect."
  });
```

Hahanapin ang user using normalized email. bcrypt.compare checks the submitted password against the stored hash. Same error text ang unknown email at wrong password, para hindi directly sabihin kung registered ang email.

Code reference: server/routes/api.js:106-112

2. JWT generation

```
jwt.sign({ sub: user.id }, process.env.JWT_SECRET,
  { expiresIn: "7d" })
```

sub means subject: ID ng account. Signed ang JWT para ma-detect ang tampering. **Hindi ito encryption**; huwag maglagay ng secrets sa payload. JWT_SECRET stays on the server. May readiness check na at least 32 characters ang secret; hindi nito automatic ginagarantiya na random o strong ang value.

Code reference: server/routes/api.js:33-40, 76-85

3. Cookie protection

httpOnly: true - hindi mababasa ng ordinary browser JavaScript ang cookie.
sameSite: "lax" - nililimitahan ang cookie sa maraming cross-site request scenarios.
secure in production - HTTPS required para ipadala ang cookie sa production.
maxAge: 604800000 - 7 days sa milliseconds; JWT expiration is also 7 days.

Hindi ibig sabihin ng HttpOnly na immune sa XSS ang app. At SameSite is a protection layer, hindi proof na lahat ng CSRF scenarios ay covered.

Code reference: server/routes/api.js:76-86

4. Protected requests, reload, at logout

Automatic cookie transport

withCredentials: true tells axios to include credentials where browser cookie rules allow it. Hindi mano-manong kumukuha ng JWT ang React. cookieParser() lets Express read req.cookies. Walang Bearer-token header flow sa implementation na ito.

Code reference: client/src/services/api.ts:2-7; server/app.js:13-19

Backend gate

```
const payload = jwt.verify(
  req.cookies.bizz_session || "", process.env.JWT_SECRET);
req.user = await User.findById(payload.sub);
if (!req.user) throw new Error();
next();
```

After public auth routes, nilalagay ang middleware na ito bago ang protected routes. verify checks the signature and expiration. Then kinukuha ang actual user sa database. Missing/invalid/expired cookie or missing user leads to 401. Hindi sapat ang pag-edit ng frontend state para malampasan ito.

Code reference: server/routes/api.js:114-132

Kapag ni-refresh ang page

App checks /health, then calls /me and /plans. /me returns publicUser(req.user), kaya backend-verified identity ang ginagamit sa UI. Pag walang valid session, 401 ang response. Ang local demo/browser storage ay hiwalay na guest experience, hindi authentication bypass sa protected API.

Code reference: client/src/App.tsx:48-55, 93-120; server/routes/api.js:226; client/src/services/api.ts:29-35

Logout

POST /api/auth/logout calls res.clearCookie("bizz_session"). Frontend logout calls that endpoint and clears signed-in workspace state. **Limitation:** clearing this cookie does not revoke an already copied JWT on the server. Walang token denylist or per-session revocation in these routes.

Code reference: server/routes/api.js:114-116; client/src/App.tsx:245-259

5. Access control at security boundaries

Owner filtering: hindi lang login ang protection

```
Plan.findOne({ _id: id, owner: req.user.id })
```

Server-derived req.user.id ang owner, hindi user ID na basta galing sa form. A valid plan ID alone is not enough. Kapag hindi pag-aari ng current user, hindi ibabalik ang record. Ganito rin ang approach sa conversations at saved calculations.

Code reference: server/routes/api.js:233, 251-259; server/routes/api.js:135, 162; server/routes/calculations.js:12, 37-50

Rate limit at request origins

Register at login share an auth limiter: 30 requests per 15 minutes, using the library's default IP-based identification. Hindi ito per-account lockout. networkAccess checks Origin against same-origin or CLIENT_ORIGINS and allows credentialed CORS for permitted origins. Requests without Origin are not rejected by this origin check, so CORS is not authentication.

Code reference: server/routes/api.js:43-48, 94, 106; server/middleware/http.js:12-40

Status codes na dapat maintindihan

201: created account. **200**: successful login/logout or data read.
400: invalid fields/body. **401**: invalid credentials/session.
403: rejected origin. **404**: record unavailable or not owned.
409: duplicate record. **429**: rate limit reached.
503: database or auth configuration not ready.

Code reference: server/middleware/http.js:28-29, 42-60; server/routes/api.js:33-48, 94-130, 259 onward

Do not overclaim sa presentation

Wala pang email verification, forgot-password/reset flow, MFA/OTP, OAuth/Google login, refresh-token rotation, or server-side JWT revocation sa inspected auth implementation. Hindi rin ito role-based admin system. Deployment needs correct HTTPS, origin configuration, and secret management. Existing tests are not a full penetration test.

6. Demo script at panel Q&A

Suggested 2-minute walkthrough

1. Open AuthModal and show required fields. Explain client vs server validation.
2. Show register route and point to bcrypt.hash(..., 12).
3. Show login route: database lookup then bcrypt.compare.
4. Show session(): JWT subject, expiry, and cookie flags.
5. Show jwt.verify middleware, then owner-filtered Plan query.
6. Explain reload via /me and logout via clearCookie.

"Bakit hindi localStorage ang token?"

"HttpOnly cookie ang gamit namin, kaya hindi directly accessible sa JavaScript ang session token. Automatic itong kasama sa eligible browser requests. May remaining XSS/CSRF considerations pa rin."

"Paano kapag pinalitan ang plan ID sa URL?"

"The server filters by both record ID and authenticated owner ID. Frontend navigation is not the security boundary."

"Bakit may JWT pero may database lookup pa?"

"JWT verifies the signed identity claim. The user lookup confirms that the account still exists and loads the current profile. Stateless token signing does not mean zero database queries."

"Ano ang proof na gumagana?"

server/api.test.js covers registration and /me, wrong-password rejection, logout causing /me to return 401, successful re-login, and record ownership isolation. The latest suite had 13 passing tests using disposable MongoDB. Hindi ito proof ng current production uptime or complete security certification.

Code reference: server/api.test.js:36-50, 346-355; owner isolation cases in the same file; server/routes/calculations.js:37-50

Quick recap

Validate -> hash/compare -> sign cookie -> verify each request -> filter by owner. Ito ang five-part explanation ninyo. Keep credentials, JWT secrets, and real user passwords off presentation slides and screenshots.